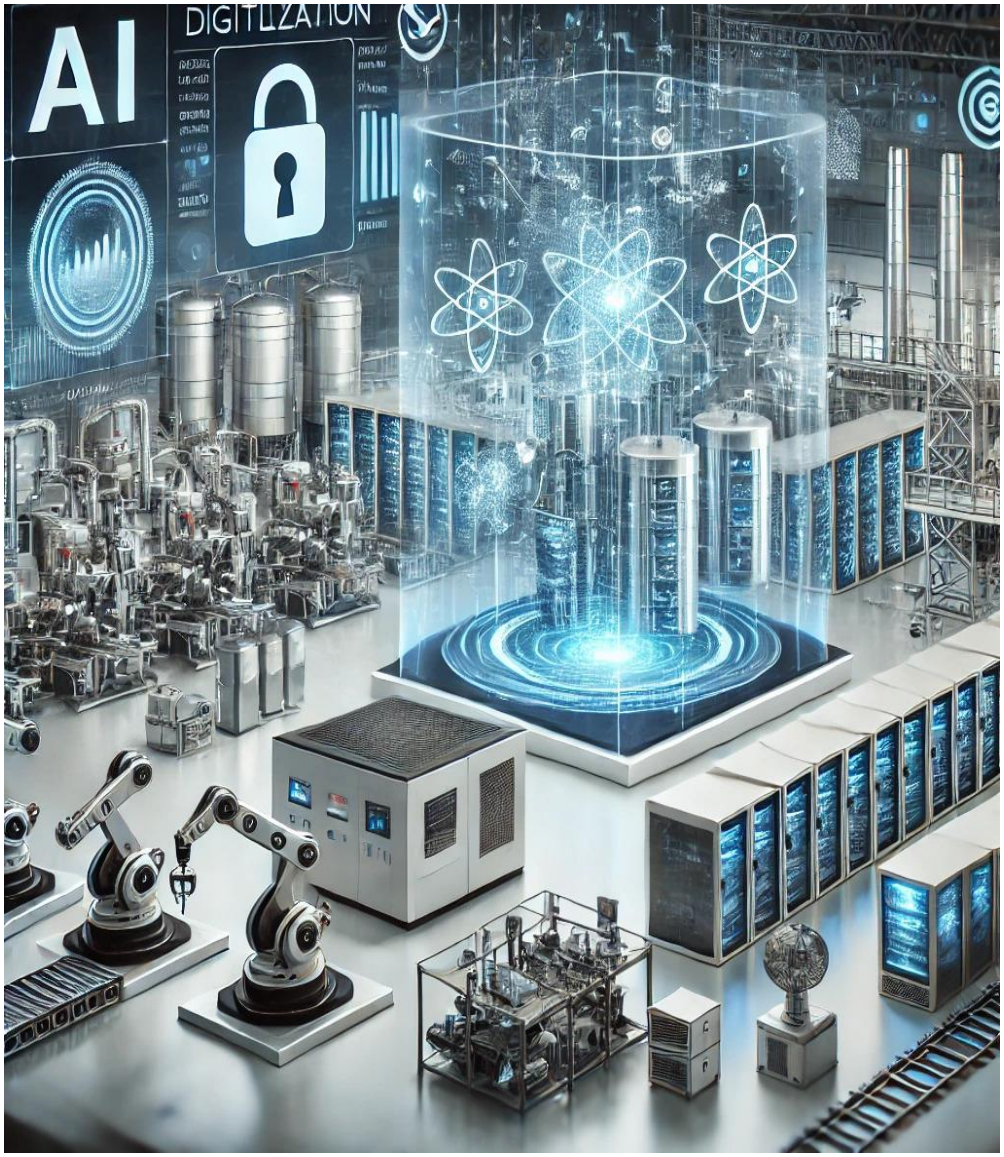


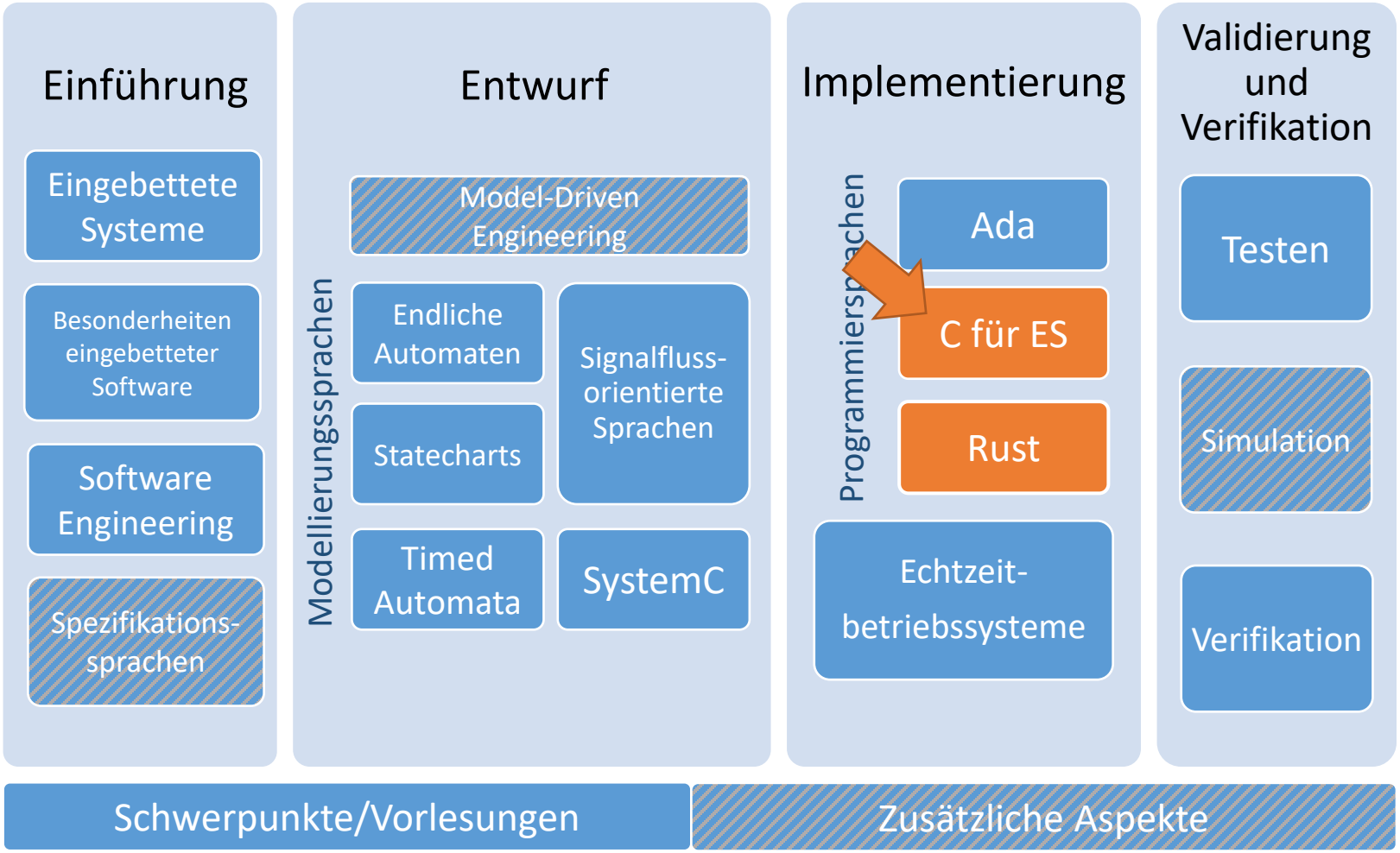
Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Embedded Systems Engineering

C für Embedded und Rust

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

Diese Vorlesung



Teil 1: C für eingebettete Systeme

C für ES: Inhalt

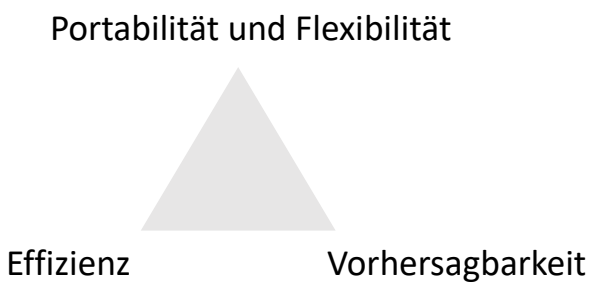
- Motivation
- Einleitung
- Hardwarezugriff
- POSIX
  - Überblick
  - POSIX Threads
  - Mutexe und Semaphore
- MISRA C
- Fazit

Motivation

Bis jetzt: Fokus auf Modellierung. Ab jetzt: Implementierung

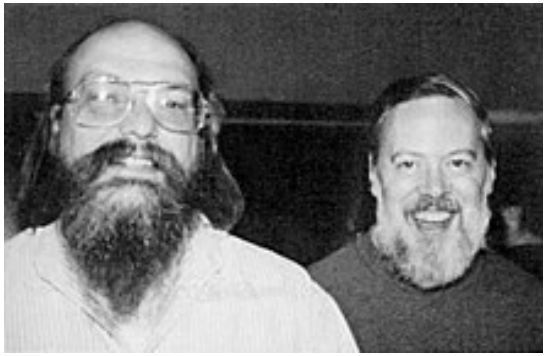
Programmiersprachen für eingebettete Systeme müssen besonderen **Anforderungen** genügen:

- Programmierung **zeitabhängiger und nebenläufiger** Programmabläufe
- **Synchronisations-** und **Kommunikationsmechanismen**
- Kontrolle über die genaue Abfolge der Ausführung von Prozessen (**Scheduling**)
- **Vorhersagbarkeit** des Programmverhaltens
- Optimale **Ressourcennutzung**



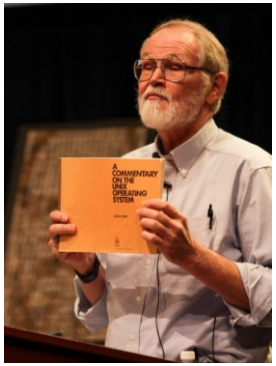
Programmiersprache C

- Entwickelt von Brian Kernighan und Dennis Ritchie
- Entstand Anfang der 70er Jahre bei AT&T Bell Labs als Programmiersprache für das **neue Betriebssystem Unix**
- Basiert auf der Programmiersprache *B*, die wiederum auf BCPL (*Basic Combined Programming Language*) basiert.
- Greift Konzepte der **strukturierten Programmierung** auf (Algol 68):
  - Kontrollstrukturen: bedingte Anweisungen (*if/then/else, switch*) und Schleifen (*for, while*)
  - Aufteilung in Unterroutinen



Ken Thompson und Dennis Ritchie

Bild lizenziert gemäß CC BY-SA



Brian Kernighan

C für eingebettete Systeme

- C ist eine **Hochsprache** → Typsystem und Kontrollstrukturen
- C ist **maschinennah**
  - Bitweise Anordnung von Datenstrukturen (alignment)
  - Zugriff auf explizite Speicheradressen über Zeiger
- **Weitere Vorteile**: minimales Laufzeitsystem, wenig Speicherbedarf zur Laufzeit

Einschränkungen im Vergleich zu anderen Sprachen:

- Keine Exceptions, Garbage Collection, Objektorientierung, …
- Keine Range Checks, keine Type Checks zur Laufzeit, eingeschränkte Typsicherheit zur Compile Time → **Sicherheitsrisiko**

Trotz dieser Einschränkungen ist C seit Jahren ungebrochen die **meistverwendete** Programmiersprache **für eingebettete Systeme.**

Teil 1: C für ES

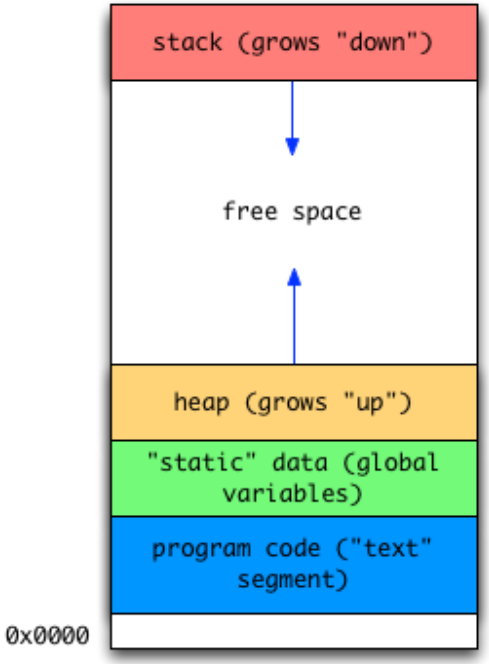
- Motivation + Einleitung
- Hardwarezugriff
- POSIX
  - Überblick
  - POSIX Threads
  - Mutexe und Semaphore
- MISRA C
- Fazit

Speicher in Modernen Betriebssystemen

Physischer Speicher: physische Ressourcen, auf denen Daten gespeichert werden können und auf die der Computer zugreifen kann: Cache, RAM, Festplatte, …

Virtueller Speicher:

- Adressierbarer Raum, auf den Programmcode zugreifen kann. Wird jedem Prozess zugeordnet.
- Abstraktion des OS: wird auf verschiedene Teile des physischen Speichers abgebildet
- Zwei zentrale Komponenten:
  - **Stack**: enthält z.B. lokale Variablen, Funktionsparameter, Rücksprungadressen. Speicher muss nach LIFO freigegeben werden → beschränkte Use Cases, aber schnell
  - **Heap**: enthält dynamisch zugewiesenen Speicher (mit `malloc/free`)



https://jsommers.github.io/cbook/pointersarrays.html

Pointer in C

- Pointer: zeigt auf eine Adresse im Speicher
- Werden primär zum Lesen und Schreiben von Variablen verwendet

Zwei wichtige Operatoren:

- **Inhaltsoperator** *: gibt Wert zurück, der an Pointeradresse gespeichert ist
- **Adressoperator** &: gibt Adresse zurück, an der Datenobjekt (z.B. Variable) gespeichert ist

Beispiel:

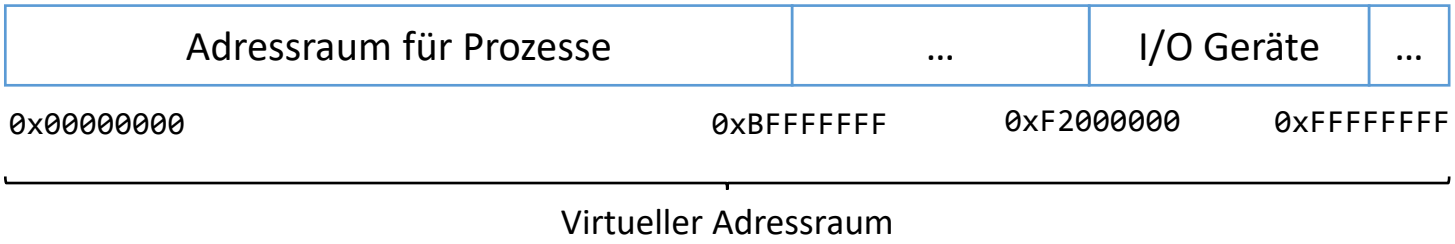
```
int x = 42;
int *pointer = &x;
printf("Adresse *pointer=%p, Wert *pointer=%d\n", &pointer, *pointer);
```

Interaktion mit Peripherie

Normalerweise interagieren wir mit Peripherie über Gerätetreiber.
Aber: **wie können wir ohne Treiber aus C Code mit Peripherie interagieren?**

- **Lösung**: Zugriff über Lesen/Verändern von Werten im Speicher

Beispiel: Broadcom BCM2835 (Raspberry PI 1 und 2)



Quelle: https://www.raspberrypi.org/app/uploads/2012/02/BCM2835-ARM-Peripherals.pdf

Zugriff auf Peripherie mit Pointer (schematisch):

```
unsigned short int *gpio;
gpio = (volatile unsigned short int*)0xF2000001;
*gpio = 0x1;
```

- GPIO Pin ist über Adresse `0xF2000001` erreichbar
- Schalte GPIO Pin auf *High*, indem Wert 1 in Register schreiben

Teil 1: C für ES

- Motivation + Einleitung
- Hardwarezugriff
- POSIX
  - Überblick
  - POSIX Threads
  - Mutexe und Semaphore
- MISRA C
- Fazit

Der Standard POSIX 1003.1

C bietet keine eigenständigen Echtzeitgarantien, keine Nebenläufigkeit

- Schnittstellen zur (Echtzeit-) Systemfunktionalität: **POSIX-Standard**

POSIX = *Portable Operating System Interface (for UniX)*

Definiert **Schnittstellen zwischen Betriebssystem und Applikation:**

- Systemschnittstellen zur **portablen Programmierung** von Anwendungen
- Echtzeit-Erweiterungen
- Thread-Erweiterungen

- Portable Schnittstellendefinitionen, trotzdem existiert eine Reihe von Parametern, die unterschiedlich interpretiert werden können
- Definition von Echtzeit im Standard: *„Echtzeitverhalten in eingebetteten (Betriebs-) Systemen beschreibt die Fähigkeit des Systems, die geforderten Dienste in einer beschränkten Antwortzeit zu erbringen.“*

Teil 1: C für ES

- Motivation + Einleitung
- Hardwarezugriff
- POSIX
  - Überblick
  - POSIX Threads
  - Mutexe und Semaphore
- MISRA C
- Fazit

POSIX Threads

- **Threads** sind **nebenläufige Teile** eines Programms, die auf einen **gemeinsamen Speicherbereich** zugreifen und auf **einem oder mehreren Prozessoren** ausgeführt werden.

- Der POSIX-Standard definiert eine **einheitliche API** für das Erzeugen und Manipulieren von Threads (unabhängig von der Implementierung)

- Bibliotheken, die diese Threads implementieren: *pthread libraries*
- Implementierungen für UNIX und Windows
- Nicht Teil des ISO-C-standards, sondern in *libthread.so*
- unter Linux in der glibc mittlerweile implementiert
- Kompilieren z.B. mit:

```
gcc -lpthread -o myThreadedProgram myThreadedProgram.c
```

Wichtig für ES

- In Bare-Metal-Systemen gibt es keine pthreads
- In RTOS (FreeRTOS, Zephyr etc.) gibt es eigene Thread-APIs
- pthreads setzen ein POSIX-kompatibles OS voraus

POSIX Threads API

Zentrale Bestandteile:

`pthread_t`

- Datentyp für einen Thread

```
pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                void* (*start_routine) (void*), void *arg)
```

- Initialisiert einen Thread und seine Ausführung
- `attr` – Legt bestimmte Eigenschaften des Threads fest
- `start_routine` – Die Hauptfunktion des Threads. Nimmt `void*` als Argument an und gibt `void*` als Ergebnis zurück.
- `arg` – Pointer zu dem Argument, mit dem `start_routine` aufgerufen werden soll.

```
pthread_join(pthread_t thread, void**retval)
```

- Wartet, bis der Thread fertig ist
- `retval` – Pointer zur Variable, in der das Ergebnis des Threads gespeichert werden soll. Das Ergebnis NULLwird nicht gespeichert.

POSIX Threads Beispiel

```c
#include <stdio.h>
#include <pthread.h>

void *print_char (void *ch)
{
  int i;
  for (i=0; i<10; i++)
    printf ("%c", *(char*)ch);
  return NULL;
}

int main ()
{
  char ch1='-', ch2='*';
  pthread_t p1, p2;
  pthread_create (&p1, NULL, print_char, &ch1);
  pthread_create (&p2, NULL, print_char, &ch2);
  pthread_join (p1, NULL);
  pthread_join (p2, NULL);
  printf ("\n");
  return 0;
}
```

Gib das Zeichen *ch* 10x aus.

Erzeuge Thread *p1* mit Hauptfunktion *print_char* und Zeichen *ch1* als Argument.

Warte, bis *p1* terminiert.

Welche Ausgabe(n) erzeugt das Programm?

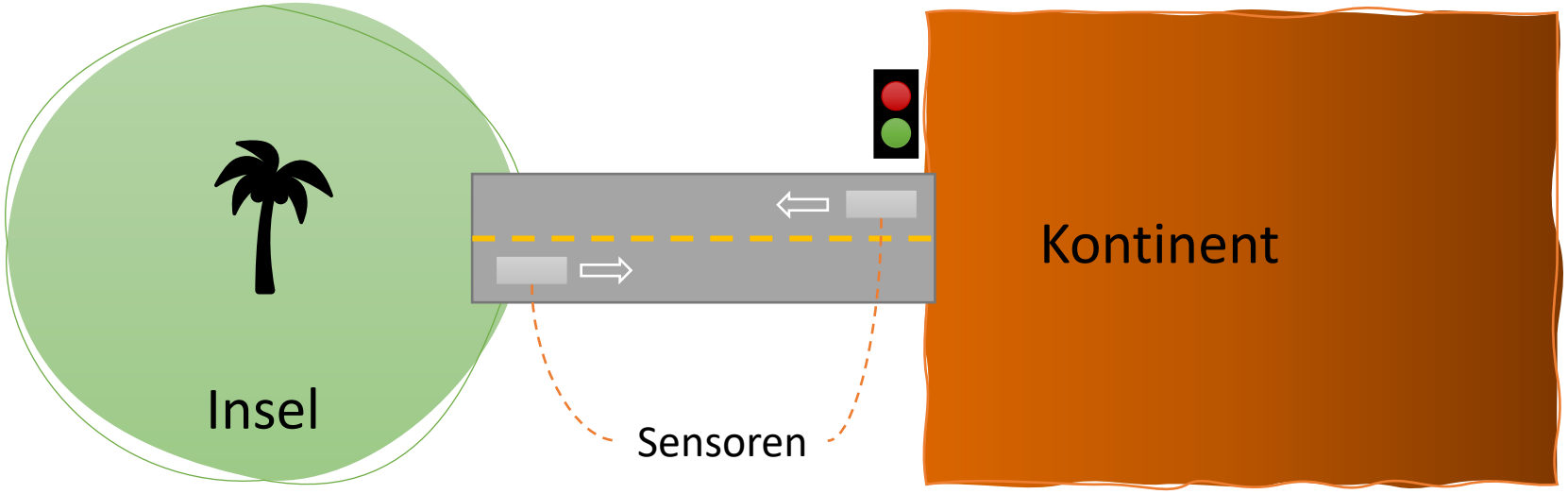
Teil 1: C für ES

- Motivation + Einleitung
- Hardwarezugriff
- POSIX
  - Überblick
  - POSIX Threads
  - Mutexe und Semaphore
- MISRA C
- Fazit

Beispiel: Inselzugang

Eine Insel ist durch einer Brücke mit dem Kontinent verbunden. Es dürfen **maximal 1000 Autos** auf der Insel sein.

Um die Begrenzung durchzusetzen gibt **es eine Ampel** am Eingang zur Insel und **Sensoren** an beiden Brückenauffahrten.



Inselzugang mit POSIX Threads

```
enum Farbe {ROT, GRUEN};
void set_ampel(enum Farbe neue_farbe);
int anzahl_autos;
void *sensor_in, *sensor_out;

int main () {
    pthread_t thread_in, thread_out;
    pthread_create(&thread_in, NULL, check_eingang, NULL);
    pthread_create(&thread_out, NULL, check_ausgang, NULL);
    pthread_join(thread_in, NULL); pthread_join(thread_out, NULL);
}
```

```
void* check_eingang(void *arg) {
            while (true) {
        await_sensor(sensor_in);
        anzahl_autos += 1;
        if (anzahl_autos >= 1000) {
            set_ampel(ROT);
        } }
}
```

```
void* check_ausgang(void *arg) {
            while (true) {
        await_sensor(sensor_out);
        anzahl_autos -= 1;
        if (anzahl_autos < 1000) {
            set_ampel(GRUEN);
        } }
}
```

Verhält sich das Programm immer wie gefordert?

Inselzugang: Kritische Abschnitte

```
enum Farbe {ROT, GRUEN};
void set_ampel(enum Farbe neue_farbe);
int anzahl_autos;
void *sensor_in, *sensor_out;
…
```

```
void check_eingang() {
    while (true) {
        await_sensor(sensor_in);
        anzahl_autos += 1;
        if (anzahl_autos >= 1000) {
            set_ampel(ROT);
        } }
}
```

```
void check_ausgang() {
    while (true) {
        await_sensor(sensor_out);
        anzahl_autos -= 1;
        if (anzahl_autos < 1000) {
            set_ampel(GRUEN);
        } }
}
```

Der Zugriff auf die globale Variable *anzahl_autos* ist nicht geschützt:

- Die Ampel kann rot zeigen, obwohl Platz auf der Insel ist
- Die Ampel kann grün zeigen, obwohl die Insel voll ist

Threadsicherheit

- Zugriff auf gemeinsame Ressourcen (z.B. Dateien oder globale Variablen) muss synchronisiert werden (keine ***race conditions***)
- Wenn Funktionen in Threads aufgerufen werden, muss sichergestellt sein, dass diese ***thread-safe*** sind (= frei von *race conditions*)

Synchronisationsmechanismen im POSIX-Standard: Mutexe und Semaphore

- Stellen sicher, dass sich nur eine bestimmte Anzahl von Threads gleichzeitig in einem kritischen Programmabschnitt befindet.

Mutexe

Ein **Lock/Mutex** kann kritische Abschnitte schützen.

- Mutex = „mutual exclusion“

Auch von POSIX unterstützt:

`pthread_mutex_t`

- Datentyp für einen Mutex

```
pthread_mutex_init(pthread_mutex_t *mutex,
                   const pthread_mutexattr_t* attr)
```

- Initialisiert den Mutex

`pthread_mutex_lock(pthread_mutex_t *mutex)`

- Wartet bis der Mutex frei ist. Holt sich dann den Mutex für den aktuellen Thread.

`pthread_mutex_unlock(pthread_mutex_t *mutex)`

- Gibt den Mutex frei

Inselzugang mit Mutex

```
enum Farbe {ROT, GRUEN};
void set_ampel(enum Farbe neue_farbe);
int anzahl_autos;
void *sensor_in, *sensor_out;
pthread_mutex_t ampel_lock;
…
```

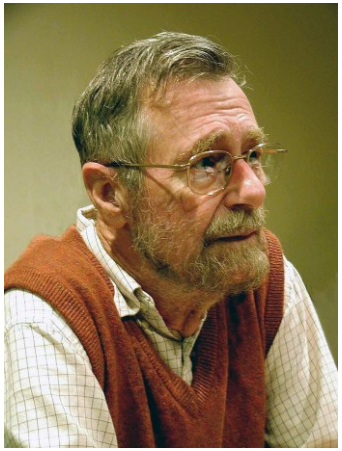
```
void check_eingang() {
    while (true) {
        await_sensor(sensor_in);
        pthread_mutex_lock(&ampel_lock);
        anzahl_autos += 1;
        if (anzahl_autos >= 1000) {
            set_ampel(ROT);
        }
        pthread_mutex_unlock(&ampel_lock);
    }
}
```

```
void check_ausgang() {
    while (true) {
        await_sensor(sensor_in);
        pthread_mutex_lock(&ampel_lock);
        anzahl_autos -= 1;
        if (anzahl_autos < 1000) {
            set_ampel(GRUEN);
        }
        pthread_mutex_unlock(&ampel_lock);
    }
}
```

Durch den Mutex kann sich der Wert von *anzahl_autos* nicht ändern, während ein Thread diesen schreibt und liest.

Semaphore

- In den 1960er Jahren von Edsger Dijkstra als Zähler für „verfügbare Ressourcen“ konzipiert
- Zwei **atomare Operationen**:
  - *wait*: dekrementiere Zähler, wenn Zählerwert größer Null. Sonst warte. *(= Ressource nehmen)*
  - *signal*: inkrementiere Zähler *(= Ressource freigeben)*
- Mutexe werden häufig mit Semaphoren implementiert



Quelle: https://en.wikipedia.org/wiki/File:Edsger_Wybe_Dijkstra.jpg

Auch von POSIX unterstützt:

`sem_t` Datentyp für einen Semaphor

`sem_init(sem_t *sem, int pshared, unsigned int value)`
Initialisiert den Semaphor mit Initialwert *value*

`sem_wait (sem_t *sem)`

`sem_post(sem_t *sem)`

Inselzugang mit Semaphor

```
enum Farbe {ROT, GRUEN};
void set_ampel(enum Farbe neue_farbe);
int anzahl_autos;
void *sensor_in, *sensor_out;
sem_t ampel_lock;
sem_init(&ampel_lock, 0, 1);
…
```

1 Thread darf in kritischen Abschnitt

```
void check_eingang() {
    while (true) {
        await_sensor(sensor_in);
        sem_wait(&ampel_lock);
        anzahl_autos += 1;
        if (anzahl_autos >= 1000) {
            set_ampel(ROT);
        }
        sem_post(&ampel_lock);
    }
}
```

```
void check_ausgang() {
    while (true) {
        await_sensor(sensor_in);
        sem_wait(&ampel_lock);
        anzahl_autos -= 1;
        if (anzahl_autos < 1000) {
            set_ampel(GRUEN);
        }
        sem_post(&ampel_lock);
    }
}
```

Wir können den Mutex durch einen Semaphor mit Maximalwert 1 ersetzen.

Alternativer Inselzugang mit Semaphor

```
enum Farbe {ROT, GRUEN};
void set_ampel(enum Farbe neue_farbe);
int anzahl_autos;
void *sensor_in, *sensor_out;
sem_t ampel_lock;
sem_init(&ampel_lock, 0, 1000);
…
```

1000 Autos dürfen auf die Insel

```
void check_eingang() {
    while (true) {
        await_sensor(sensor_in);
        sem_wait(&ampel_lock);
        allow_car;
    }
}
```

```
void check_ausgang() {
    while (true) {
        await_sensor(sensor_in);
        sem_post(&ampel_lock);
    }
}
```

1 Platz frei

Anderes Verhalten:

- Ampel standardmäßig rot.
- Auto fährt an Eingang. Wenn Kapazität: einzelnes Auto bekommt grün, bis durchgefahren.

Teil 1: C für ES

- Motivation + Einleitung
- Hardwarezugriff
- POSIX
  - Überblick
  - POSIX Threads
  - Mutexe und Semaphore
- MISRA C
- Fazit

MISRA C

- C-Standard aus der **Automobilindustrie** (*Motor Industry Software Reliability Association*)
- Richtlinien zur Steigerung der Qualität sicherheitskritischer Systeme
- Erste Version 1998, aktuell: MISRA C:2012
  - Zusätzlich: MISRA Compliance:2016 (Anleitung zur Einhaltung des MISRA Standards)
- Richtlinien in drei **Prioritätskategorien** aufgeteilt:
  - *Mandatory*: keine Abweichungen erlaubt
  - *Required*: Abweichungen müssen formal beantragt werden
  - *Advisory*: Abweichungen erlaubt, müssen dokumentiert werden
- **Tools** zur Prüfung der *MISRA conformance* verfügbar

Ausgewählte MISRA C Richtlinien

Einschränkung von Sprachkonstrukten, z.B.:

- Vermeidung von Compilerunterschieden (z.B. anstatt int INT16 oder nur /*…*/ Kommentare)

Konventionen für verbesserte Struktur, z.B.:

- Rule 14.8 (required): The statement forming the body of a switch, while, do … while or for statement shall be a compound statement.(enclosed within braces)

Best-Practice Regeln, z.B.:

- Rule 3.1 (required): All usage of implementation-defined behaviour shall be documented
- Rule 12.13 (advisory): The increment (++) and decrement (--) operators should not be mixed with other operators in an expression.

C für ES: Fazit

- C ist seit Jahren die **weitverbreitetste** Programmiersprache für eingebettete Systeme
- C bietet Typsystem und Kontrollstrukturen einer **Hochsprache** und Maschinennähe einer **Low-Level** Sprache. Es ist **effizient** und auf nahezu jeder Plattform verfügbar.
- Der **POSIX** Standard definiert **Schnittstellen** zu (Echtzeit-) Systemfunktionalität. **Mutexe** und **Semaphore** erlauben sichere Zusammenarbeit nebenläufiger Threads.
- **MISRA C** ist ein Standard der Automobilindustrie zur Steigerung der Qualität sicherheitskritischer Systeme.
- Trotz aller Vorteile haben C Programme **gravierende Nachteile**. Sie sind fehleranfällig, aufwendig zu debuggen und besitzen keinen portablen Binärcode. **Alternativen**: Rust, Ada, …

Teil 2: Rust

Rust: Inhalt

- Einführung
- Sprachübersicht
- Speichersicherheit durch Ownership + Borrowing
- Nebenläufigkeit und Kommunikation
- Fazit + Literatur

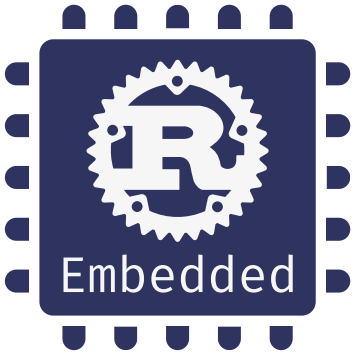
Entstehung von Rust

- In den 2000ern waren Firefox und Internet Explorer mit Abstand die am weitesten verbreiteten Browser
- 2008 wurde Chrome von Google veröffentlicht
  - **Erster parallelisierter Browser**, der sich **Multikernprozessoren** zunutze machte
  - **Grobe Parallelisierung**: jede Webseite in einem nebenläufigen Prozess
  - **Feinere Parallelisierung theoretisch möglich**: mehrere Aufgaben einer Webseite nebenläufig ausgeführt
    - Parsen vom HTML, Formgestaltung durch CSS, JavaScript Interpreter…
    - **Wegen Fehleranfälligkeit** der C++ Implementierung allerdings **impraktikabel**
- Ab 2010 **entwickelte Mozilla** eine neue Programmiersprache namens **Rust um eine feinere Parallelisierung zu ermöglichen**
- Die ersten in Rust programmierten Komponenten wurden 2017 in Firefox integriert, mit einer großen Verbesserung der Performance

Embedded Rust

Verschiedene Eigenschaften von Rust sind für **sicherheitskritische** und **nebenläufige** Systeme relevant:

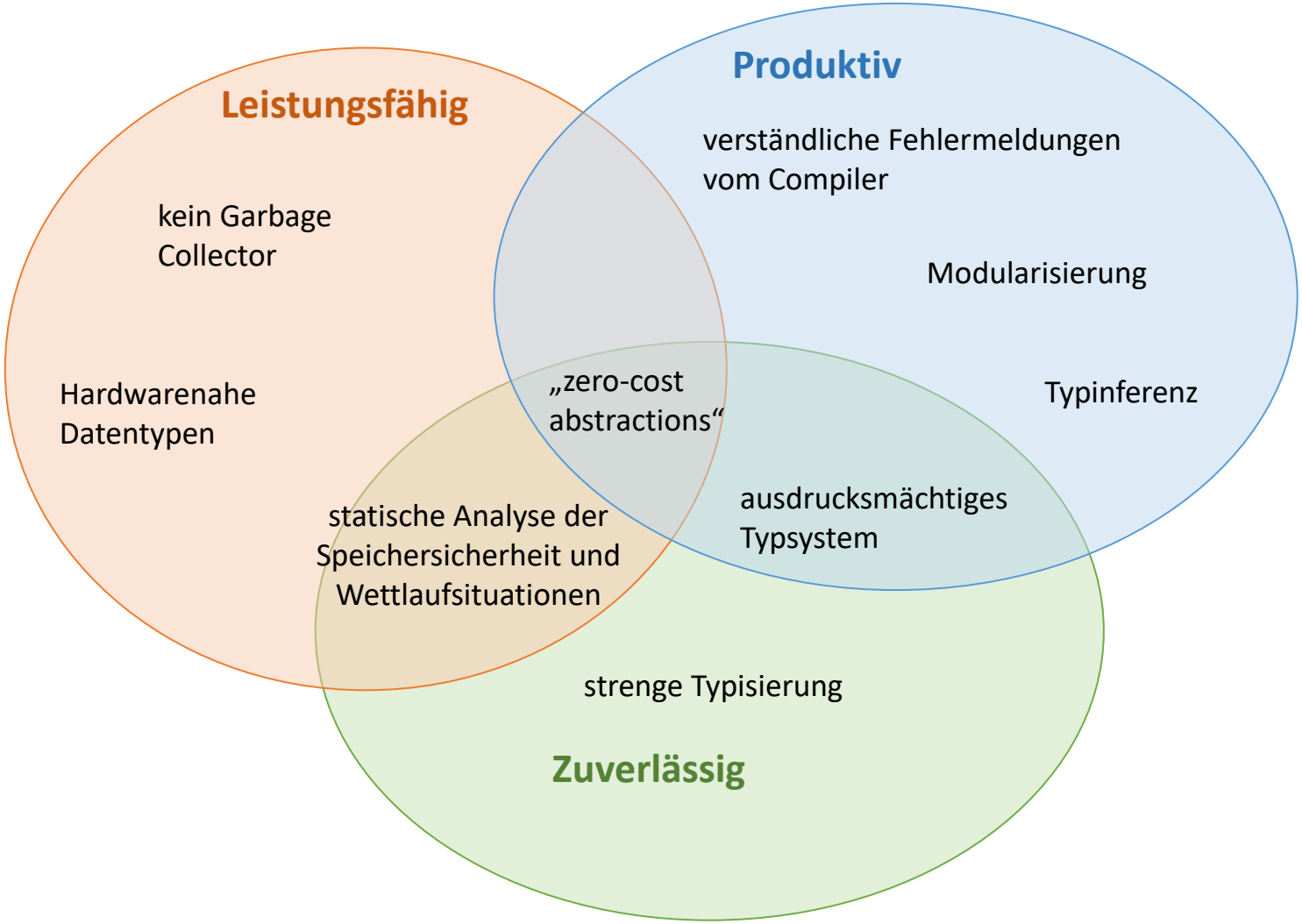
- Fehleranfällige Muster, die in C von Sicherheitsrichtlinien (z.B. MISRA C) verboten sind, werden vom Compiler geprüft
- Speichersicherheit statisch abgesichert
- Race Conditions statisch vermieden
- Fehlkonstruktionen (z.B. Nullpointer) können durch das Typsystem ausgeschlossen werden



https://rust-embedded.github.io/blog/

Embedded Rust ist noch in der **Entwicklungs-/Forschungsphase**, wird aber von einer sehr **aktiven Community** vorangetrieben

Eigenschaften von Rust



Vor- und Nachteile von Rust

Rust vs. C oder Java:

- In jeder Programmiersprache kann man sichere Programme schreiben (und in jeder unsichere)
- Rust vermeidet einige *Fußangeln* der C-Sprachen → weniger Fehler, bessere Les- und Wartbarkeit
- Ist sehr viel systemnäher als Java

- Die in Rust integrierten Konzepte von *Ownership* und *Borrowing* sind in anderen Sprachen extrem nützlich, auch wenn sie nicht statisch geprüft werden
- Rust ist ziemlich neu und im eingebetteten Bereich noch wenig verbreitet
- Da Rust ziemlich neu ist, gibt es **weniger Tools und Bibliotheken** als in anderen Programmiersprachen, allerdings eine **sehr engagierte Community**

Wird bereits in verschieden Firmen wie Dropbox, Yelp und Deliveroo angewandt (Quelle: https://www.rust-lang.org/production/users)

Beliebteste Sprache bei der jährlichen StackOverflow-Befragung seit 2016!

Teil 2: Rust

- Einführung
- Sprachübersicht
- Speichersicherheit durch Ownership + Borrowing
- Nebenläufigkeit und Kommunikation
- Fazit + Literatur

Was für eine Sprache ist Rust?

Rust hat eine C-ähnliche Syntax, verwendet aber viele Konzepte aus funktionalen Programmiersprachen:

- Präferenz für **unveränderbare Variablen**
- **Keine Nullzeiger**
- Ausdrucksmächtige *Enums* mit **Pattern Matching**
- **Strenge Typisierung mit Typinferenz** ähnlich ML
- **Generische Typen und „Schnittstellen“** ähnlich Haskell

Rust: Funktionen und Datentypen

Variablen sind standardmäßig unveränderbar
→ Vermeidung kritischer Sektionen
mut kennzeichnet veränderbare (mutable) Variablen

Slices: Referenzen zu Arrays unbekannter Größe.

Tupel

```
fn su    und_anzahl(zahlen: &[i32]) -> (i32, i32)
{
      let mut summe: i32 = 0;
      let mut anzahl: i32 = 0;
      for x in zahlen.iter() {
            summe += x;
            anzahl += 1;
      }
       (summe, anzahl)
}
```

numerische Datentypen mit verschiedenen Größen (i8, i16…)

die Typen der Variablen können vom Compiler abgeleitet werden

Der letzte Ausdruck in der Funktion bestimmt den Rückgabewert

Rust: Typinferenz

```
fn summe_und_anzahl(zahlen: &[i32]) -> (i32, i32) {
        let mut summe = 0;
        let mut anzahl = 0;
        for x in zahlen.iter() {
                summe += x;
                anzahl += 1;
        }
        (summe, anzahl)
}
```

Die Typen der Variablen können vom Compiler abgeleitet werden

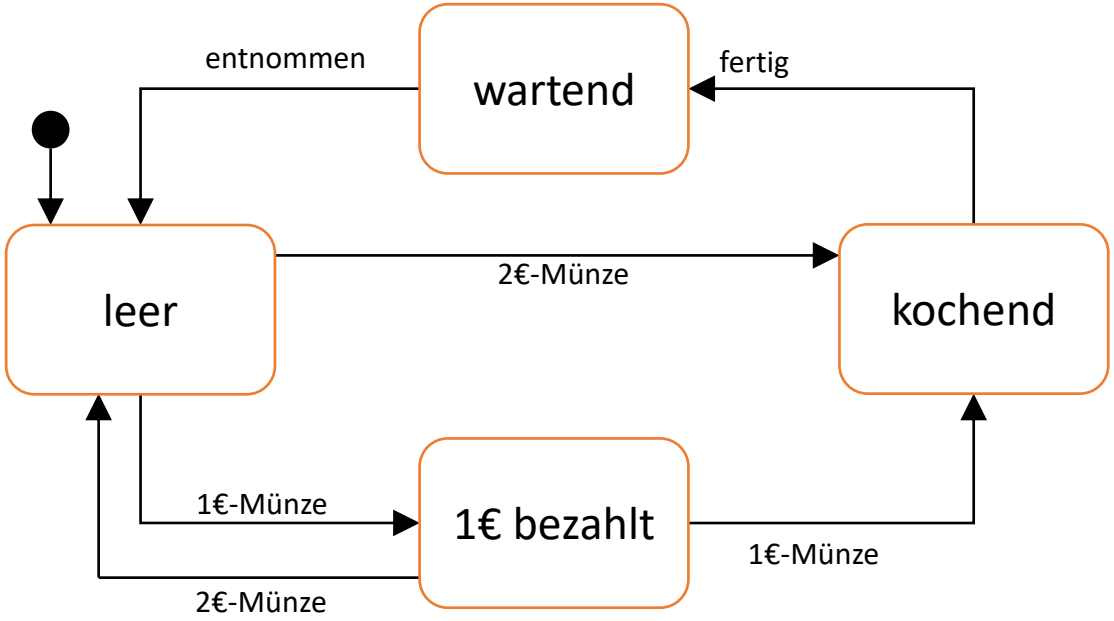
Rust: Enums

```
enum Muenze { FuenfzigCent, EinEuro, ZweiEuro }
```

```
fn wert_in_cents(m: Muenze) {
    match m {
        FuenfzigCent => 50,
        EinEuro => 100,
        ZweiEuro => 200
    }
}
```

Ähnlich zu switch in C.
Stellt aber sicher, dass alle
Fälle behandelt werden

Beispiel: Kaffeeautomat



Kaffeeautomat mit Enums

```
enum Muenze { FuenfzigCent, EinEuro, ZweiEuro }
enum Zustand { SammeltGeld(i32), Ko
enum Ereignis { Einwurf(Muenze), Ka
```

Enum-Werte können weitere Werte anderer Typen beinhalten

```
fn update(zustand: Zustand, ereignis: Ereignis) -> Zustand {
    match (z                    gnis) {
        (Koch                   affeeFertig) => WartetMitBecher,
        (KochtKaffee, _) => KochtKaffee,
        (WartetMitBecher, BecherEntnommen) => SammeltGeld(0),
        (WartetMitBecher, _) => WartetMitBecher,
        (SammeltGeld(betrag), Einwurf(m)) => {
            let neuerBetrag = betrag + m.wert();
            if neuerBetrag < 200 {
                SammeltGeld(neuerBetrag)
            } else {
                KochtKaffee
            }
        }
        (SammeltGeld(betrag), _) => zustand
    }
}
```

Wildcard

Erstes Match wird genommen

Bestimmte Werte (z.B. betrag) sind dann nur in entsprechenden Fällen verfügbar

Methoden in Rust

```
enum Muenze { FuenfzigCent, EinEuro, ZweiEuro }

enum Schein { FuenfEuro, ZehnEuro, ZwanzigEuro }
```

```
impl Muenze {
    fn wert(self) -> i32 {
        match self {
            FuenfzigCent => 50,
            EinEuro => 100,
            ZweiEuro => 200
        }
    }
}
```

```
impl Schein {
    fn wert(self) -> i32 {
        match self {
            FuenfEuro => 500,
            ZehnEuro => 1000,
            ZwanzigEuro => 2000
        }
    }
}
```

```
let m = FuenfzigCent;

let s = ZehnEuro;

let betrag = m.wert() + s.wert();
  // => 1050
```

Methode wird durch Namen und Typ der entsprechenden Variablen **während der Übersetzung erkannt**.

Traits und generische Funktionen

```
// …

trait Geld {
    fn wert(self) -> i32;
}

impl Geld for Muenze {
    fn wert(self) -> i32 {
        match self {
            FuenfzigCent => 50,
            EinEuro => 100,
            ZweiEuro => 200
        }
    }
}

impl Geld for Schein {
    // …
}
```

Traits ähneln *Interfaces* in objektorientierten Sprachen

Generische Funktionen ähneln C++-Templates, können aber die Implementierung von Traits voraussetzen

```
fn gesamtbetrag<T : Geld>(liste : &[T]) -> i32 {
    let mut summe = 0;
    for geldeinheit in liste.iter() {
        summe += geldeinheit.wert();
    }
    summe
}

gesamtbetrag([FuenfEuro, ZehnEuro, ZehnEuro])
  //  => 2500
```

„zero-cost“ Abstraktion: wird beim Übersetzen aufgelöst und trägt nicht zu Laufzeit oder Speicherverbrauch bei

Teil 2: Rust

- Einführung
- Sprachübersicht
- Speichersicherheit durch Ownership + Borrowing
- Nebenläufigkeit und Kommunikation
- Fazit + Literatur

Speichersicherheit

Bei der Speicherverwaltung müssen bestimmte Eigenschaften **gewährleistet** werden:

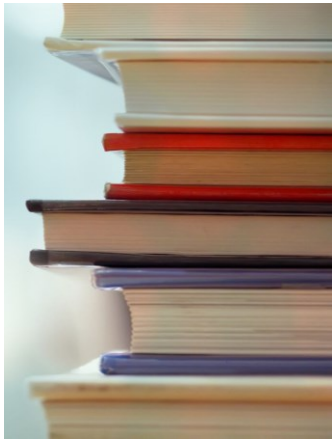
- Nur initialisierte Werte lesen
- Speicherbereiche müssen nach Gebrauch freigestellt werden. *Aber nur dann, wenn keine weiteren Referenzen auf dem Bereich sind!*

Dynamische Sicherung durch *Garbage Collection*:

- Programm wird von der Laufzeitumgebung unterbrochen, um übrige Speicherbereiche freizustellen
- Schwer bei nebenläufigen Prozessen
- Unvorhersagbare Laufzeit, ungeeignet für Echtzeitsysteme

Speichersicherheit durch Ownership + Borrowing

Analogie: Buch verschenken vs. Ausleihen



- Ein Buch hat einen Besitzer (**Owner**). Der Besitzer kann mit dem Buch machen was er möchte (z.B. Notizen hineinschreiben, das Buch weitergeben oder wegwerfen)
- Wenn man ein Buch verschenkt, bekommt es einen neuen Besitzer.
- Wenn man es jemandem verleiht (Borrowing), ändert sich der Besitzer nicht
  - Derjenige, der das Buch ausgeliehen hat, darf es **nicht verändern**.
  - Während das Buch ausgeliehen ist, kann der Besitzer es nicht verändern.
  - Wenn ich ein Buch besitze, kann ich mehrere **Kopien** davon verleihen
  - Wenn ich das Buch jemandem ausleihe um es signieren zu lassen, dann ändert sich der Inhalt des Buches und meine Kopien sind ungültig

Statische Prüfung der Speichersicherheit - Ownership

In Rust wird **Speichersicherheit statisch geprüft**

Ownership: für jeden Speicherbereich ist **exakt eine Variable verantwortlich**

- **Verantwortung darf übertragen werden** (z.B. bei einer Zuweisung oder einem Funktionsaufruf), **danach kann auf die Variable nicht mehr zugegriffen werden (Move)** (außer die Funktion gibt die Verantwortung zurück, mittels Rückgabewert)
- Wird die Verantwortung nicht übertragen und der **Geltungsbereich der Variable verlassen, lässt das Programm den Speicher frei**
- Variablen können auch kopiert werden: die Variable wird kopiert und die Ownership nicht abgegeben, Voraussetzung: Implementierung des *Copy traits* (standardmäßig bei primitiven Datentypen wie Integer)

Statische Prüfung der Speichersicherheit - Borrowing

Borrowing: Referenzen zum Speicherbereich dürfen **befristet „ausgeliehen“** werden, solange die Referenz den **Geltungsbereich der verantwortlichen Variablen nicht überdauert**

- Speicherbereich wird im Empfänger nicht freigegeben, sondern Ownership zurückgegeben
- Regel: entweder *beliebig viele* unveränderbare Referenzen ODER *genau eine* veränderbare Referenz

Ownership und Borrowing auch **für andere Ressourcen anwendbar**!

- Dateien, die geschlossen werden müssen
- Schreibberechtigung auf Variablen, um **Wettlaufsituationen zu vermeiden**
- Referenzen zu **mit Mutex geschützten Daten**

Verantwortung und Übergabe

```
fn main() {
    let s1 = String::from("Welt");
    let gruss = begruessen(s1);
    println!("{}", gruss);
    println!("{}", s1);  // Fehler!
}


fn begruessen(name : String) -> String {
    format!("Hallo, {}!", name);
}
```

1) Variable s1 ist für den Speicherbereich des String verantwortlich

2) Verantwortung wird an gruss übergeben

4) Nach der Übergabe darf s1 nicht mehr verwendet werden

3) Am Ende des Geltungsbereiches von name wird der entsprechende Speicher freigestellt

Ausgeliehene Referenzen

```
fn main() {
    let s1 = String::from("Welt");
    let name = &s1;
    let gruss = begruessen(&s1);
    println!("{}", gruss);
    println!("{}", name);
}

fn begruessen(name : &String) -> String {
    format!("Hallo, {}!", name)
}
```

Es dürfen mehrere Referenzen gleichzeitig vorhanden sein

String wird an die Funktion „ausgeliehen“

Funktion darf den String nicht verändern!

Befristete Ausleihe

```
fn main() {
    let name;
    {
        let s1 = String::from("Welt");
        name = &s1;
        let gruss = begruessen(&s1);
        println!("{}", gruss);
    }
    println!("{}", name);
}

fn begruessen(name : &String) -> String {
    format!("Hallo, {}!", name)
}
```

Lebensdauer des Strings in s1

Die Referenz ist noch sichtbar, trotz Freistellung des Strings
→ Vom Compiler erkannt und verboten

Veränderbare Referenzen

```
fn main() {
    let mut s1 = String::from("Welt");
    begruessen(&mut s1);
    println!("{}", s1);

    let refs1 = &s1;
    begruessen(&mut s1); // Fehler!
}

fn begruessen(s1 : &mut String) {
    *s1 = format!("Hallo, {}!", s1);
}
```

String wird für die Funktion „veränderbar ausgeliehen“

Wenn eine veränderbare Referenz existiert, darf keine andere Referenz existieren!
→ kritische Abschnitte vermieden

Teil 2: Rust

- Einführung
- Sprachübersicht
- Speichersicherheit durch Ownership + Borrowing
- Nebenläufigkeit und Kommunikation
- Fazit + Literatur

Nebenläufigkeit und Kommunikation

Nebenläufigkeit über Threads, ähnlich wie POSIX

Mehrere Ansätze zur **Kommunikation** zwischen Threads:

- Über gemeinsamen Speicher, geschützt durch Mutexe
- Über Kanäle mit der *Crossbeam*-Bibliothek

Beispiel: Kommunikation über Kanäle

```
fn device_driver(r: Receiver<char>, s : Sender<char>) {
    loop {
        device_data = read_device_data();
        select! {
            recv(r) -> user_data => {
                write_to_device(user_data);
            },
            send(s, device_data) => {},
            default(Duration::from_secs(10)) => {
                // timeout exception handling
            }
        }
    }
}
```

Rust: Fazit

- Rust wurde für **effiziente nebenläufige Systeme** entwickelt, findet auch im eingebetteten Bereich Anwendung
- Erlaubt **hardwarenahe** Programmierung
- **Strenges und ausdrucksmächtiges Typsystem**, das trotzdem **Typinferenz** unterstützt
- Statisch geprüfte Sicherheitseigenschaften mittels **Ownership und Borrowing**
- Keine Objektorientierung, sondern gemeinsame Schnittstellen mittels **Traits**

Rust: Literatur

- Steve Klabnik , Carol Nichols. The Rust Programming Language https://doc.rust-lang.org/book/
- The Embedded Rust Book, *Embedded Rust Working Group*
  https://rust-embedded.github.io/book/
- Crossbeam-Bibliothek https://github.com/crossbeam-rs/crossbeam
- https://rust-lang.org/